

Полное подробное объяснение лабораторной работы 4

«Модули и функции»

Учебное пособие для выполнения и защиты

Документ подготовлен как подробное руководство: теория, разбор каждого задания, пояснение по стеку, советы по запуску и защите.

Как пользоваться этим документом

Это не просто пересказ условия, а полноценное руководство. Его можно читать тремя способами.

1. Если нужно понять лабораторную с нуля — читайте главы 1–5 подряд. Там объясняются модули, соглашения о вызовах, стек, регистры, пролог/эпилог, variadic-функции, вызовы scanf/printf и различия между System V, Microsoft 64 и cdecl32.
2. Если нужно выполнить конкретный номер — переходите сразу к разделу с нужным заданием. Каждый номер разобран по схеме: цель, что нужно знать, как делать, на что обратить внимание, что рисовать на стеке, что говорить на защите.
3. Если нужно подготовиться к сдаче — обязательно прочитайте разделы «Как рисовать стек», «Типичные ошибки», «Чек-лист перед сдачей» и «Что обычно спрашивают на защите».

Важно: этот документ объясняет логику выполнения и защиты. Он не заменяет исходное задание и не отменяет необходимость соблюдать именно ваш вариант, вашу структуру проекта и ваши рисунки стека.

1. Что это за лабораторная и зачем она нужна

Лабораторная работа 4 посвящена теме «Модули и функции». На практике это означает, что студент должен научиться связывать вместе код на C/C++ и ассемблере, понимать соглашение о вызовах функции, правильно передавать параметры и возвращаемые значения, а также корректно организовывать стековый кадр функции.

Если сформулировать совсем просто, то основная идея лабораторной такая: программа может состоять из нескольких модулей; часть модулей может быть написана на C/C++, часть — на ассемблере; чтобы эти модули работали вместе, они обязаны одинаково понимать ответы на вопросы «куда класть аргументы», «откуда брать результат», «кто обязан сохранять регистры», «как должен выглядеть стек в момент вызова».

Именно это и задаётся соглашением о вызовах, или calling convention / ABI. В этой работе вы отрабатываете этот принцип несколько раз на очень разных примерах: простая целочисленная функция, функция с double, чисто ассемблерная программа с puts, более сложная программа со scanf/printf и локальными переменными, вариант с классическим прологом и бонусная функция с большим количеством аргументов.

2. Теория, без которой ЛР4 не получится

2.1. Что такое модуль

Модуль — это отдельный исходный файл. Например, main.cpp, unit.cpp, main.S, unit.S. Во время сборки каждый модуль компилируется отдельно, а затем все объектные файлы связываются компоновщиком в одну программу.

Если функция описана в одном модуле, а вызывается из другого, компилятор и компоновщик должны знать две вещи:

- какое у функции имя на уровне объектного файла;
- какой у неё прототип: тип результата, типы аргументов и соглашение о вызовах.

2.2. Что такое extern и extern "C"

Когда функция реализована в ассемблерном модуле, в C++-коде её нужно объявить как внешнюю. Но для C++ этого мало: язык C++ выполняет decoration / name mangling, то есть преобразует имена

функций на уровне компоновки. Ассемблерный модуль обычно экспортирует обычное имя, например `f1`, а C++ без специальных указаний будет искать украшенное имя.

Поэтому в C++ нужно писать `extern "C"`. Это означает: не применять C++-декорирование, а использовать C-совместимое имя. Тогда имя функции в C++ и ассемблере совпадёт, и компоновщик сможет связать вызов с реализацией.

```
extern "C" int f1(int x, int y);
extern "C" double f2(double x, double y);
```

2.3. Что такое соглашение о вызовах

Соглашение о вызовах — это набор правил, который определяет:

- в каких регистрах или по каким адресам в стеке передаются аргументы;
- где возвращается результат;
- какие регистры можно портить, а какие нужно сохранять;
- каким должен быть стек перед вызовом другой функции;
- кто очищает стек после вызова;
- как обрабатываются функции с переменным числом параметров.

В ЛР4 нужно работать как минимум с двумя 64-битными соглашениями:

- System V amd64 psABI — типичное соглашение для GNU/Linux, BSD, macOS x64;
- Microsoft 64 — соглашение для Windows x64.

Дополнительно, если доступна 32-битная среда, можно реализовать и `cdecl` для x86.

2.4. Стек: что это и как о нём думать

Стек — это область памяти, которая растёт в сторону уменьшения адресов. Это значит: чем «глубже» стек, тем меньше адрес. Визуально в ЛР стек просят рисовать так: адреса растут снизу вверх, а вершина стека находится внизу рисунка. Это непривычно, но очень важно для правильных схем.

Обычно внутри стека находятся:

- адрес возврата;
- сохранённые регистры (если функция их сохраняет);
- локальные переменные;
- временные данные;
- стековые аргументы для вызова вложенной функции.

Когда функция вызывается инструкцией `call`, процессор автоматически помещает в стек адрес возврата. После этого функция может дополнительно уменьшить `rsp/esp`, чтобы зарезервировать место под свои локальные данные.

2.5. Пролог и эпилог функции

Пролог — это начальные команды функции, которые подготавливают стековый кадр. Эпилог — завершающие команды, которые восстанавливают стек и возвращают управление.

В этой лабораторной используются два варианта.

- Укороченный пролог/эпилог: функция просто делает `sub rsp, N` в начале и `add rsp, N` в конце; локальные переменные адресуются относительно `rsp`.
- Классический пролог/эпилог: функция сначала сохраняет `rbp/ebp`, затем копирует туда `rsp/esp`, а локальные переменные адресуются относительно `rbp/ebp`.

```
; укороченный пролог
subq $40, %rsp
...
```

```

addq $40, %rsp
ret

; классический пролог
pushq %rbp
movq %rsp, %rbp
subq $80, %rsp
...
leave
ret

```

Классический пролог длиннее, но удобнее для отладки и для стабильной адресации переменных: смещения относительно `rbp` не меняются по ходу функции. Укороченный пролог короче и чуть проще с точки зрения количества команд, но адресовать данные относительно `rsp` иногда менее наглядно.

2.6. Какие регистры бывают важны в этой работе

В лабораторной надо различать как минимум четыре группы регистров.

- Регистры общего назначения для целых значений и адресов: `eax/rax`, `ecx/rcx`, `edx/rdx`, `esi/rsi`, `edi/rdi`, `r8`, `r9` и т. д.
- Стековые регистры: `esp/rsp` — вершина стека; `ebp/rbp` — база кадра.
- SSE/AVX-регистры `xmm/ymm` для чисел с плавающей точкой.
- Регистры, которые по ABI нужно сохранять, и регистры, которые можно свободно менять.

Важная идея: то, что регистр существует, ещё не означает, что его можно бездумно использовать. Если ABI требует сохранить регистр, а функция его портит и не восстанавливает, это считается нарушением соглашения, даже если программа «случайно работает» на ваших тестах.

2.7. Возврат результата

Для целочисленных и указательных результатов обычно используется `eax/rax`. Для `double` и `float` — `xmm0`. В 32-битном x86 для некоторых вариантов floating point может использоваться стек `x87` (`st(0)`), если код написан через классические инструкции FPU. Ваша конкретная реализация и целевой ABI определяют, что именно нужно использовать.

Отсюда следует правило: при проектировании ассемблерной функции нужно сразу знать не только откуда взять аргументы, но и куда положить результат перед `ret`.

2.8. Выравнивание стека

Один из самых важных источников ошибок — неверное выравнивание стека перед вызовом другой функции. Например, `printf`, `scanf` и `puts` ожидают, что стек будет выровнен согласно ABI. Если нарушить это правило, программа может падать, вести себя нестабильно или просто считаться неверной на защите.

Поэтому при рисовании стека надо учитывать не только локальные переменные, но и тот факт, что перед вложенным вызовом стек должен соответствовать правилам именно текущего соглашения.

2.9. Variadic-функции: почему `scanf` и `printf` — особый случай

`scanf` и `printf` принимают переменное число аргументов. Это сложнее, чем обычный вызов функции с фиксированным числом параметров, потому что компилятор и программист должны особенно аккуратно соблюдать ABI.

Ключевые идеи:

- в `scanf` передаются адреса переменных, потому что функция должна записать значения по этим адресам;
- в `printf` передаются уже сами значения;
- `float` в `variadic`-контексте продвигается до `double`;
- в некоторых ABI нужно дополнительно сообщить, сколько XMM-регистров использовано для передачи аргументов с плавающей точкой.

Именно поэтому задания №4 и №5 считаются самыми содержательными и одновременно самыми трудными в ЛР4.

2.10. SSE и AVX: что нужно знать для задания с `double`

Задание с числами двойной точности требует использовать либо AVX-инструкции `vsubsd/vdivsd`, либо SSE-аналоги `subsd/divsd`. Здесь не нужно знать весь набор SIMD-команд. Достаточно понимать, что такие инструкции умеют выполнять операции над `double` в `xmm/ymm`-регистрах.

В типовом случае логика очень простая: входные `double` уже находятся в `xmm`-регистрах; нужная команда выполняет операцию; результат остаётся в `xmm0` и возвращается вызывающему коду.

2.11. Чем отличаются System V amd64 psABI, Microsoft 64 и `cdecl32`

Параметр	System V amd64	Microsoft 64	<code>cdecl32</code>
Платформа	Linux/macOS/BSD x64	Windows x64	x86 32-bit
Первые целые аргументы	<code>rdi, rsi, rdx, rcx, r8, r9</code>	<code>rcx, rdx, r8, r9</code>	через стек
Первые FP-аргументы	<code>xmm0-xmm7</code>	<code>xmm0-xmm3</code>	обычно через стек
Кто чистит стек	стек восстанавливается по кадру вызывающего кода	аналогично, но <code>shadow space</code> обязателен	<code>caller</code> очищает стек
Shadow space	нет	есть, 32 байта перед вызовом	нет
Возврат <code>int/pointer</code>	<code>rax</code>	<code>rax</code>	<code>eax</code>
Возврат <code>double</code>	<code>xmm0</code>	<code>xmm0</code>	зависит от реализации, часто <code>x87 st(0)</code>
Базовая особенность	много регистровых аргументов	меньше регистровых аргументов + <code>shadow space</code>	классическая стековая передача

3. Общая стратегия выполнения ЛР4

4. Сначала прочитать условие и выписать, какие именно проекты и под какие ABI нужно сделать.
5. Для каждого задания понять, где проходит граница между C/C++ и ассемблером: только одна функция на ассемблере или вся программа целиком.
6. До написания кода определить, откуда берутся аргументы и куда возвращается результат.
7. Если в задаче есть вложенный вызов `puts/scanf/printf` — до написания кода нарисовать стек или хотя бы черновую схему.
8. После написания кода проверить сборку, запуск, корректность результата и совпадение поведения C/C++-версии и ассемблерной версии там, где это требуется.
9. После этого подготовить чистовые рисунки стека на бумаге.

Это важный совет: в ЛР4 очень опасно сначала писать код «как получится», а потом пытаться объяснить стек задним числом. Намного надёжнее сначала продумать ABI и стек, а уже потом реализовывать программу.

4. Разбор задания №1: ассемблерная функция f1 для целого выражения

Вариант	Формула
1	$f1(x, y) = -7 + x + 8y$
2	$f1(x, y) = 12 + x + y$

Суть задания

Нужно написать ассемблерную функцию $f1(x, y)$, вычисляющую целое выражение от двух целых аргументов, и вызвать её из программы на C/C++.

Варианты

См. таблицу или пояснение в тексте раздела.

Что здесь проверяется

- понимание передачи целочисленных аргументов по ABI;
- понимание возврата целочисленного результата;
- умение связать C/C++ и ассемблер через extern "C";
- умение сделать простую, но корректную leaf-function без нарушения соглашения.

Как мыслить при решении

Надо ответить на четыре вопроса: где находятся x и y ; как вычислить выражение минимальным числом команд; куда положить результат; нужен ли вообще стековый кадр. Во многих реализациях стековый кадр не нужен, потому что функция ничего не сохраняет и никого не вызывает.

Почему условие упоминает lea

Инструкция lea удобна для выражений вида $base + index * scale + const$. В варианте $x + 8y - 7$ она позволяет красиво вычислить адресоподобную арифметику без обращения к памяти.

Что должно быть в C/C++-части

- прототип через extern "C";
- передача тестовых аргументов;
- вызов f1;
- желательно — расчёт того же выражения на C/C++ для проверки.

Что рисовать на стеке

Если функция не создаёт стековый кадр, рисунок минимален: адрес возврата. Для cdecl32 дополнительно нужно показать стековые аргументы x и y .

Что могут спросить на защите

- почему здесь может не понадобиться пролог;
- почему нужен extern "C";
- в чём различие между System V и Microsoft 64;
- почему результат возвращается через eax/rax;

- чем удобен lea.

5. Разбор задания №2: ассемблерная функция f2 для double

Вариант	Формула
1	$f2(x, y) = x - y$
2	$f2(x, y) = x / y$

Суть задания

Нужно написать функцию от двух double, используя AVX или SSE, и вызвать её из программы на C/C++.

Варианты

См. таблицу или пояснение в тексте раздела.

Что проверяется

- понимание передачи параметров с плавающей точкой;
- работа с xmm-регистрами;
- возврат результата через xmm0;
- различие fixed-arity и variadic-вызовов.

Что надо знать

Для fixed-arity функции два double в 64-битных ABI обычно приходят в xmm-регистрах. Значит, функция может просто выполнить subss/divss или vsubss/vdivss и вернуть xmm0.

Почему реализация почти одинаковая в двух 64-битных ABI

Потому что для небольшого числа double обе ABI используют регистровую передачу. Отличия ярче проявляются, когда аргументов много, типы смешаны или функция variadic.

Что рисовать на стеке

Если функция не использует стек и не вызывает другие функции, рисунок снова минимален. Для cdecl32 аргументы double становятся стековыми и занимают больше места.

Что объяснять на защите

- почему double передаются через xmm-регистры;
- почему результат идёт через xmm0;
- чем отличаются SSE и AVX на уровне записи команды;
- почему cdecl32 меняет картину вызова.

6. Разбор задания №3: чисто ассемблерная программа с puts

Суть задания

Вся программа, включая main, пишется на ассемблере и вызывает puts из libc.

Что важно

- правильно передать адрес строки;
- подготовить стек по правилам ABI перед call puts;
- понимать, что pure-asm main ничем не освобождает от соблюдения соглашения о вызовах.

Как различается вызов puts

System V amd64: адрес строки в rdi. Microsoft 64: адрес строки в rcx плюс обязательный shadow space. cdecl32: адрес строки передаётся через стек.

Что рисовать на стеке

Рисунок особенно важен именно перед вызовом puts. Там должно быть видно и выравнивание, и служебные области.

Что спрашивают на защите

- почему в Windows нужен shadow space;
- почему важно выравнивание стека;
- почему main на ассемблере всё равно обязан соблюдать ABI.

7. Разбор задания №4: scanf/printf и укороченный пролог

Имя	Тип	Размер
i16	short	2 байта
i32	int	4 байта
i64	long long	8 байт
f32	float	4 байта
f64	double	8 байт

Суть задания

Нужно сначала написать C/C++-версию с пятью локальными переменными разных типов, затем реализовать аналог полностью на ассемблере, используя укороченный пролог.

Какие переменные

См. таблицу или пояснение в тексте раздела.

Что проверяется

- размещение локальных переменных в стеке с учётом выравнивания;
- различие между fixed и variadic-вызовами;
- понимание, что scanf получает адреса, а printf — значения;
- проектирование стекового кадра по байтам.

Почему сначала нужна C/C++-версия

Она служит эталоном по поведению, формату строк и вводу-выводу. После этого ту же логику повторяют на ассемблере.

Как мыслить о размещении переменных

Удобно сначала разместить i64 и f64 как полные 8-байтовые объекты, затем объединить i32 и f32 в одном 8-байтовом слове, а i16 положить в отдельное слово с padding.

Почему scanf и printf нельзя рисовать одним рисунком

Потому что в scanf передаются адреса переменных, а в printf — уже значения. Кроме того, для printf участвуют variadic promotions.

Что такое variadic promotion

float в variadic-контексте участвует как double, поэтому аргументы printf для f32 нужно готовить именно как double.

Как делать задание на практике

- написать и проверить C/C++-версию;
- разложить переменные по стеку на бумаге;
- отдельно рассчитать вызов scanf;
- отдельно рассчитать вызов printf;
- подобрать общий размер кадра;
- реализовать asm-main строго по рисунку.

Что обязательно уметь объяснить

- почему i16 нельзя класть по произвольному адресу;
- почему i32 и f32 удобно совместить;
- почему перед scanf нужны адреса;
- почему float перед printf становится double;
- как выбирался общий размер кадра.

8. Разбор задания №5: тот же смысл, но классический пролог

Суть задания

Программа аналогична №4, но теперь используется классический пролог/эпилог и адресация локальных переменных через rbp/ebp.

Что меняется по сравнению с №4

- появляется сохранённый rbp/ebp;
- смещения локальных данных относительно rbp/ebp отрицательные;
- на рисунке нужно показать, куда указывает rbp/ebp;
- нужно уметь назвать смещения и от rsp, и от rbp.

Почему классический пролог удобен

Он даёт стабильную опорную точку кадра. Смещения локальных переменных от rbp/ebp не зависят от временных изменений вершины стека.

Что важно показать на рисунке

- адрес возврата;
- старый rbp/ebp;
- новый rbp/ebp, указывающий на слово со старым rbp/ebp;
- локальные переменные;
- стековые аргументы scanf/printf.

Что спрашивают на защите

- чем классический пролог лучше укороченного;
- почему локальные переменные теперь адресуются через rbp;
- что изменилось на рисунке по сравнению с №4;
- почему смещения от rbp отрицательные.

9. Разбор задания №6: бонусная функция с восемью параметрами

Суть задания

Нужно написать на C/C++ функцию с восемью параметрами `int`, которая печатает их и возвращает восьмой параметр, а затем вызвать её из `asm-main`.

Что проверяется

- понимание границы между регистровыми и стековыми аргументами;
- подготовка большого вызова;
- понимание `cdecl32` с множественными `push`;
- комбинация `asm-main` и C/C++-function.

Что надо помнить по ABI

System V amd64: первые 6 целых аргументов — регистровые. Microsoft 64: первые 4 — регистровые. `cdecl32`: все аргументы стековые.

Что могут спросить

- какие аргументы стековые в каждой ABI;
- кто очищает стек в `cdecl32`;
- почему в Microsoft 64 даже при стековых аргументах нужен `shadow space`.

10. Как рисовать стек правильно

Общие правила рисунка

- стек рисуется столбцом прямоугольников;
- для 64-битных вариантов слово стека — 8 байт;
- для `cdecl32` — 4 байта;
- адреса растут снизу вверх;
- вершина стека находится внизу рисунка;
- если в одном слове несколько объектов, границы надо показать внутри слова.

Алгоритм построения рисунка

- определить момент: после пролога или перед конкретным вызовом;
- найти адрес возврата;
- посмотреть размер кадра;
- выписать смещения локальных переменных;
- добавить стековые аргументы и служебные области;
- только после этого рисовать.

Масштаб рисунка

Удобный практический масштаб: для 64-битного слова — 4 клетки по высоте, для 32-битного слова — 2 клетки. Тогда `i32` в 64-битном слове занимает половину, а `i16` — четверть.

Что чаще всего рисуют неправильно

- забывают адрес возврата;
- путают направление роста адресов;
- рисуют регистровый аргумент как стековый;
- забывают `shadow space`;
- не учитывают старый `rbp` в классическом прологе;

- рисуют одинаковый стек перед scanf и printf.

11. Как запускать и собирать работу

Microsoft 64

Эта часть выполняется в Windows x64. Проекты можно собирать локально через IDE или консольный toolchain, если он соответствует целевой платформе.

System V amd64

Эта часть делается в Linux/macOS/BSD или совместимой среде, например WSL. Важно запустить код именно в ABI-совместимой среде.

cdecl32

Для бонусной 32-битной реализации нужна x86-сборка, обычно через -m32 и наличие 32-битных библиотек/мультилибов.

Почему важны расширения .S и .s

.S обычно проходит через препроцессор, .s — нет. Это влияет на сборку проекта.

Что помнить про Qt Creator

Файлы .S обычно приходится вручную перечислять в SOURCES проекта.

12. Типичные ошибки и за что снижают баллы

Основные ошибки

- не сделана одна из обязательных ABI-версий;
- ассемблерная функция нарушает соглашение о вызовах;
- неверно организован стек перед puts/scanf/printf;
- забыто extern "C" и проект не линкуется;
- рисунки стека не совпадают с кодом;
- локальные переменные вынесены в сегмент данных;
- variadic-вызов реализован как обычный fixed-arity;
- float передаётся в printf без понимания promotions.

13. Чек-лист перед сдачей

Проверьте перед сдачей

- есть версии минимум под System V amd64 и Microsoft 64;
- asm-код соответствует ABI;
- где нужно, есть extern "C";
- результаты вычислений совпадают с ожидаемыми;
- scanf/printf работают на реальном вводе;
- рисунки стека совпадают с кодом;
- в Windows-проектах учтён shadow space;
- в классическом прологе показан старый rbp/ebp;
- студент умеет устно объяснить различия ABI.

14. Что обычно спрашивают на защите и как отвечать

Ключевые вопросы

- Чем отличаются System V amd64 и Microsoft 64?
- Почему нужен extern "C"?
- Почему перед printf и scanf разные рисунки стека?
- Зачем нужен классический пролог?
- Почему в Windows выделяется больше места в кадре?
- Кто очищает стек в cdecl?
- Что означает выравнивание стека?

15. Минимальные шаблоны мышления по каждому номеру

№1

Leaf-function: взять целые аргументы по ABI, вычислить выражение, вернуть результат через eax/rax.

№2

Fixed-arity function с double: взять xmm-аргументы, выполнить SSE/AVX-операцию, вернуть xmm0.

№3

Pure-asm main: подготовить стек и вызвать puts строго по ABI.

№4

Разложить 5 локальных переменных по стеку, отдельно продумать scanf и printf, использовать укороченный пролог.

№5

Повторить логику №4, но адресовать локальные через rbp/ebp и учитывать сохранённый базовый регистр.

№6

Показать понимание границы между регистровыми и стековыми аргументами при большом числе параметров.

16. Итог: что нужно реально понять, чтобы сделать и сдать ЛР4

Главная мысль

Лабораторная проверяет не умение «набирать команды», а понимание контракта между вызывающим и вызываемой функцией. Если понятны ABI, стек, регистры и variadic-вызовы, то все номера становятся логичными и объяснимыми.

Приложение А. Краткие псевдошаблоны

Leaf-function без стека

```
.globl f1
f1:
    ; взять аргументы по ABI
```

```
    ; вычислить выражение
    ; положить результат в eax/rax
    ret
```

Pure-asm main с puts

```
.globl main
main:
    ; подготовить стек по ABI
    ; передать адрес строки
    call puts
    ; вернуть 0
    ret
```

Укороченный пролог

```
main:
    subq $N, %rsp
    ; локальные переменные адресуются через %rsp
    ...
    addq $N, %rsp
    ret
```

Классический пролог

```
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $N, %rsp
    ; локальные переменные адресуются через %rbp
    ...
    leave
    ret
```

Приложение В. Что повторить перед защитой

Список

- различия System V amd64 и Microsoft 64;
- смысл extern "C";
- назначение rsp/esp и rbp/ebp;
- разницу между укороченным и классическим прологом;
- почему scanf и printf — variadic;
- как рисуется стек;
- как передаются int и double;
- что такое shadow space;
- кто очищает стек в cdecl32.